

機械語で遊ぼう (ARM64)
Let's play with ARM64

ARMプロセッサ

- 200億個以上のデバイスに内蔵されており，事実上の業界標準
- ARM社は設計図とライセンスを販売
- AV機器，自動車，カーナビ，携帯電話，オフィス機器等に利用
- IoTで注目
- さまざまなタイプのプロセッサがある．RasPi2に使われているのはARM11（Cortex-A5プロセッサ，ARMv7アーキテクチャ）
- 68系プロセッサの末裔



スパコンの富岳もMacもARM



機械語で遊ぼう

- 機械語は基本的なツール（GAS, GCC, GDB）を使えば遊べる
- ここではRasPiを配れないのでAmazon EC2 のARM64ベースのインスタンスで試そう！

機械語で遊ぼう

- 機械語は基本的なツール（GAS, GCC, GDB）を使えば遊べる
- ここではRasPiを配れないのでAmazon EC2 のARM64ベースのインスタンスで試そう！

 **Ubuntu Server 16.04 LTS (HVM), SSD Volume Type** - ami-078648cce0d33c256 (64 ビット x86) / ami-0accc21f9774a85d7 (64 ビット Arm)

Ubuntu Server 16.04 LTS (HVM), EBS General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/services>).

無料利用枠の対象

ルートデバイスタイプ: ebs 仮想化タイプ: hvm ENA 有効: はい

選択

☐ 64 ビット (x86)

☒ 64 ビット (Arm)

- docker コンテナを30個（）
- ログインの仕方

```
ssh cyber@52.68.226.54 -p 222XX
```

- XX の部分は各自に配った番号を当てはめる
- パスワードは「apple8086」

とりあえず機械語を動かしてみる

- 以下のコードをエディタで入力
- ファイル名は first.s で
- エディタは emacs, vim, nano がインストール済
- コンパイル, 実行

```
$ gcc -static -o first first.s
$ ./first
$ echo $?
```
- 2 が表示されたか確認

```
.text

.globl main

main:
    mov w0, #2
    ret
```

とりあえず足し算してみる

- 右のコードをエディタで入力
- ファイル名は add1.s で
- コンパイル, 実行

```
$ gcc -g -o add1 add1.s
```

```
$ ./add1
```

```
$ echo $?
```

- argc+1が表示されたか確認

```
.text  
.globl main
```

```
main:
```

```
add w0, w0, #1  
ret
```

```
cyber2021@b389059f67e2:~$ gcc -o add1 add1.s
```

```
cyber2021@b389059f67e2:~$ ./add1; echo $?
```

```
2
```

```
cyber2021@b389059f67e2:~$ ./add1 a b; echo $?
```

```
4
```

```
cyber2021@b389059f67e2:~$ ./add1 x y z; echo $?
```

```
5
```

Hello, world

- puts の呼び出し

1. a002.s としてコードを入力

2. gcc -g a002.s -o a002

3. ./a002 で実行

1. Call puts
2. Enter the code as a002.s
3. gcc -g a002.s
4. Execute ./a.out

```
main:
    .text
    .global main
    // puts("Hello, world!")
    adr      x0, hello
    stp      x29, x30, [sp, #-16]!
    bl       puts
    ldp      x29, x30, [sp], #16
    // exit
    mov      x8, #93
    svc      #0

hello:
    .string "Hello, world!"
    .align 2
```


関数呼び出し

関数のプロローグ（呼び出し準備）

`stp x29, x30, [sp, #-16]!`

- STP: Store Pair x29とx30を同時に保存
- x29: フレームポインタ、x30: 戻りアドレス
- `[sp, #-16]!` : プリデクレメントアドレッシング
spを16減らしてからそこに保存

関数のエピローグ（後始末）

`ldp x29, x30, [sp], #16`

- LDP: Load Pair x29とx30を同時にメモリからロード
- x29: フレームポインタ、x30: 戻りアドレス
- `[sp], #16` : ポストインクリメントアドレッシング
[sp]の位置からロードしてその後、+16加える

もう少しだけ複雑な プログラム

1 から100まで足して
結果を printf で表示

<https://gist.github.com/koide55/a511539555fc2b772cd64413ca80da68>

```
.text
.global main

main:
    mov    x1, #0
    mov    x2, #1

loop:
    cmp    x2, #100
    bgt    end
    add    x1, x1, x2
    add    x2, x2, #1
    b      loop

end:

    // display one integer
    bl     printf

    // exit
    mov    x8, #93
    svc    #0

.text
.global printf
.type     printf, %function

printf:
    stp    x29, x30, [sp, #-16]!
    adr    x0, printf_fmt
    bl     printf
    ldp    x29, x30, [sp], #16
    ret

.data
printf_fmt:    .string "%d\n"
```

gdb (今回使ってみるコマンド)

break main (b main)	ラベル main にブレークポイントを設定
info registers	すべてのレジスタの値を確認
print \$x1	x1レジスタの値を確認
stepi (si)	1インストラクション実行
b 11 if \$x2==95	11行目でレジスタx2が95になったらブレーク
cont	次のブレークポイントまで実行
finish	関数を抜けるまで実行

演習（60分）

- デバッガで break ポイントを設定し, break したところから機械語プログラムをステップ実行せよ
- デバッガでプログラムを実行中にレジスタの値がプログラム通りに変化することを観察せよ
- 複数のレジスタを使い別の計算を行うプログラムを書いて add命令, sub命令を試せ
- 端末から gdb を直接実行してみよ
- オリジナルの機械語プログラムを作り内容を説明せよ

もっとおおきい即値をレジスタにロードするには

x1に 0x1234abcd をロードして
print1d で表示させてみよう

64ビットレジスタXnで0から63、
32ビットレジスタWnで0から31
のシフト演算を行うことができる

reg, LSL, #amount

reg, LSR, #amount

reg, ASR, #amount

reg, ROR, #amount

```
.text
.global main

main:
    mov     x2, #0x1234
    mov     x3, #0xabcd
    add     x1, x3, x2, LSL #16

end:
    bl      print1d
    mov     x8, #93
    svc     #0

.text
.global print1d
.type      print1d, %function

print1d:
    stp     x29, x30, [sp, #-16]!
    adr     x0, print1d_fmt
    bl      printf
    ldp     x29, x30, [sp], #16

.data
print1d_fmt:
    .string "%lx\n"
```

Arm64の特徴 (bitmask immediate)

論理命令 (AND, ORR, EOR, ANDSなど)

特徴：即値としては任意のビット列を扱えない

特定の長さのビットパターンの繰り返しと回転で表されるもの

N : immr : imms で表現

64 bit 命令のとき

N: パターン長 (bit幅 : 32 or 64; 1bit)

immr: 回転量 (6bit)

imms: マスクビットサイズ (6bit)

`orr` `x1, xzr, #0x1010101010101010`

うまく行くパターンの例

- `#0xff00ff00ff00ff00`
- `#0xff0000ffff0000ff`
- `#0x1010101010101010`

駄目なパターンの例

- `#0xabababababababab`
- `#0x1100110011001100`

Arm64の特徴（算術命令の即値）

ADD/SUB

- 即値の範囲は12bit（0～4095）
- 12ビットシフトした形も可

```
add    x2, x0, #0xdef
add    x1, x2, #0xabc, LSL #12
```

MOV(MOVZ/MOVK/MOVN)

- 16bitの即値を指定可能
- 0/16/32/48 bit シフト可能

```
movz   x1, #0x1234
movk   x1, #0x5678, LSL #16
movk   x1, #0xabcd, LSL #32
movk   x1, #0x9999, LSL #48
```

参考リンク

arm

<https://developer.arm.com>

Arm® Architecture Reference Manual
Armv8, for Armv8-A architecture profile

ポスト「京」に備えて. . .

<http://numericalbrain.org/wp-content/uploads/armasm20170606.pdf>

arm